ABAP INTERVIEW PREPARATION

# ABAP practice workbook

Thirteen interview scenarios with model answers, a mock interview and a study plan

329 questions across 24 study segments. Reviewed 12 October 2026.

Use the short answers for oral rehearsal. The examples and pitfalls prepare you for follow-up questions. Choose the target SAP edition and release before applying an implementation detail.

Open abap-interview-studio.pages.dev online, or download the repository and open START_HERE.html for offline study. The PDF is a portable reading and printing edition.

Solve each exercise before opening the suggested answer. The scenarios are hypothetical interview exercises. Code fragments illustrate reasoning and are not activated SAP repository objects. Validate exact syntax, dependencies and release compatibility in a target SAP system before using a fragment in an application.

# Exercise 1 Choose an internal table

You receive 100,000 unique order IDs. For each of 40,000 incoming messages, you must find the order by its complete ID. In another report you must process all orders for each customer in customer order. Choose a table category and explain the trade-offs.

**Your answer before reading the solution**

## Model answer

For the 40,000 complete order-ID lookups, I would consider a hashed table with a unique order-ID key. It can find the order without checking each row. For processing orders by customer, I would use a suitable sorted key. A standard table with a sorted secondary key can also work when I must keep the original file order.

I would measure the whole job, including loading and changing the table. Extra keys need memory and maintenance. A hashed table's fast complete-key lookup does not mean searching by any partial key is equally fast.

## Follow-up

What changes when duplicate order IDs are permitted or only a partial key is known? Explain how you would handle a missing row without hiding a business error. Review ABAP-01 through ABAP-04.

# Exercise 2 Find the data-access defect

An import selects details with FOR ALL ENTRIES using an internal table of selected document IDs. The selection table can be empty. The developer also assumes duplicate result rows will be preserved.

**Your answer before reading the solution**

## Model answer

First, check whether the list of document IDs is empty. If it is, do not run this FOR ALL ENTRIES query as though it meant 'select nothing': its WHERE condition is ignored, which can return far more data within the query's remaining implicit scope. Decide explicitly what the empty-input case should do.

FOR ALL ENTRIES also removes duplicate result rows. Check which columns the query returns and whether two different business records could therefore look identical. Compare a join, a range or another supported query with the same expected result. Removing duplicate input IDs can reduce work, but only do it if it preserves the intended result.

## Follow-up

Do not promise that a join is always faster. Compare result equivalence, transferred rows, execution count and representative timings. Review the SQL segment.

# Exercise 3 Correct a CDS identity

A flight entity exposes CARRID and CONNID as its only keys while reading SFLIGHT. It associates to SCARR using the carrier ID, declares a to-many relationship and projects `_Carrier.name`. The client occasionally confuses different departure dates.

**Your answer before reading the solution**

## Model answer

Add FLDATE to the key: the same carrier and connection can have flights on different dates. Without the date, the client cannot reliably tell those departures apart. Use the actual SCARR field CARRNAME rather than guessing a field called name.

Check the association's ON condition and the data. One flight should match at most the intended carrier row; declaring a cardinality does not force the data to obey it. Likewise, a CDS key describes identity but does not create a database uniqueness check. For a new model on a release that supports it, prefer a CDS view entity and leave out the old SQL-view-name annotation.

## Follow-up

Test multiple dates, a missing carrier and unexpected duplicate target rows. Decide DCL at the actual consumed entity. Review CDS-02, CDS-03, CDS-07 and CDS-11. This replaces the malformed illustrative CDS snippet in the original PDF rather than offering it as a ready-to-activate object.

# Exercise 4 Repair RAP transaction ownership

A proposed action changes a request, attempts COMMIT ENTITIES, then would invoke a remote workflow. That nested commit violates the RAP runtime contract. The design also needs to reject invalid date ranges before persistence. The developer proposes moving the commit into the saver.

**Your answer before reading the solution**

## Model answer

Remove COMMIT ENTITIES from the action. Moving it into the saver would still be wrong: behavior code must let RAP finish the transaction. Put the date check where RAP can reject an invalid request before it is saved. Use the appropriate behavior logic for values that need to be calculated.

Start the workflow through a supported mechanism tied to the committed request. Otherwise a workflow might start for a request that later fails to save. If an external ABAP program calls EML, that outer caller checks the modification results and calls COMMIT ENTITIES. A rollback in the local database cannot undo an ordinary remote workflow call.

## Follow-up

Test invalid dates, a save failure and duplicate event delivery. A remote workflow call is not rolled back by the local database. Review RAP-04, RAP-05, BTP-07 and RICEFW-11.

# Exercise 5 Read an EML response

A validation returns an error message in REPORTED but does not fill FAILED. A determination tries to fill FAILED. A create handler returns only the database key and loses the caller's content ID.

**Your answer before reading the solution**

## Model answer

In the validation, put the rejected instance in FAILED and its error message in REPORTED. The message should identify the right instance and, where relevant, the field and path. Sending an error message alone is not the same as rejecting the change.

A determination has no FAILED response. Use a supported validation, precheck or action for the expected rejection. For a create, return the required mapping between the caller's %cid and the created instance. Keep the complete RAP instance identity in responses, including draft or preliminary-key information when the operation needs it. Otherwise the caller may not know which record a result belongs to.

## Follow-up

Explain %cid, %cid_ref, %tky and %pid without calling %tky a temporary key. For late numbering discuss preliminary-to-final mapping in ADJUST_NUMBERS. Review RAP-07 and RAP-10.

# Exercise 6 Secure an approval action

An app hides the Approve button for ordinary users. Its DCL filters one base CDS entity, but the service uses a projection without its own access control. An ordinary user can call the action URL directly.

**Your answer before reading the solution**

## Model answer

Add a backend permission check for Approve. A hidden button does not stop someone from sending the request directly. Separately use feature control to express whether approval is available in the request's current state.

Protect reads of the projection that the service actually uses. Do not assume the base entity's DCL automatically applies there; use the appropriate rules or supported explicit inheritance. Test an ordinary user reading another company's known key and calling Approve directly. Also check service access and any downstream API permissions. A value help or hidden field is not a security check.

## Follow-up

Would authorization: none or IN LOCAL MODE fix the bug? No: those deliberately bypass relevant checks in their defined contexts and require justification. Review CDS-07, ADV-02 and TEST-06.

# Exercise 7 Classify Clean Core dependencies

Classify these hypothetical cases. Assume the stated release or nomination is verified for the target system; do not infer it from a name.

| Case | Model classification and reasoning |
|---|---|
| ABAP Cloud extension using the appropriate released SAP APIs and released extension points | A for those dependencies and technologies |
| Classic code calling an API SAP has nominated as classic | B for that dependency |
| Read access to an internal SAP object that is neither released nor nominated and is not explicitly prohibited | C with upgrade risk and governance |
| Unsupported direct write to an SAP business table | D risk; replace with the supported business operation |
| Released custom wrapper calling a nominated classic API | The consumer sees a released custom contract; the underlying implementation still carries B dependency |
| BTP app using an unsupported private endpoint | Placement does not establish compliance; classify the actual dependency and remedy it |

## Follow-up

State the ATC mapping: A no corresponding usage finding, B priority 3, C priority 2, D priority 1. Explain why an ordinary security error in ATC is a different dimension. Review CORE-02 through CORE-16.

# Exercise 8 Diagnose a failed batch and stale update

An OData batch returns HTTP 200 but one inner response is 412. The UI reports success and the developer retries with If-Match set to a wildcard.

**Your answer before reading the solution**

## Model answer

Open the batch response and inspect each inner result. The outer 200 does not mean every operation succeeded. The inner 412 says a precondition failed; a stale ETag is a common cause. Read the current record and resolve the user's conflicting edit.

Do not automatically replace If-Match with a wildcard just to make the update succeed. That can remove protection against overwriting someone else's change. Check which operations belong to the same changeset or atomicity group and therefore succeed or fail together. Other groups in the same batch can have a different outcome.

## Follow-up

Explain why one batch can contain separate transaction scopes and why a CSRF token does not replace an ETag. Review ODATA-06 through ODATA-09.

# Exercise 9 Build a reliable approval flow

A committed request event is delivered twice. Approval later arrives after the requester has withdrawn the document. The callback times out after the backend accepted it.

**Your answer before reading the solution**

## Model answer

Store a stable request/event identifier so the same event does not start the workflow twice. When an approval arrives, check the request's current status, version and approver. If the request has been withdrawn, the old approval must not move it forward.

If the backend accepted a callback but its reply was lost, a retry should recognize the same decision and return the agreed result. A timeout means 'we do not yet know the outcome', not 'nothing happened'. Check the recorded outcome before repeating an irreversible action. Give support staff a way to find failed deliveries and retry only the work that is safe to repeat.

## Follow-up

Draw the boundary between the local RAP transaction, event delivery, external workflow and callback transaction. Review RICEFW-11 through RICEFW-14 and BTP-08.

# Exercise 10 Explain and test a refactoring

A report method selects data, calculates discounts, writes a file and commits the transaction. It uses a singleton for the current date and a nested SELECT in every item loop. What would you do first?

**Your answer before reading the solution**

## Model answer

First, record what the report currently does and who controls its commit. Protect those behaviors with focused tests before changing the design. Move the discount calculation into code that can be tested without selecting data or writing a file. Pass in a clock when tests need to control the date.

Measure how many database calls the nested SELECT produces. Then change the read strategy while checking that the same business rows and output remain. Keep the commit with the correct transaction owner. Test missing data, discount limits and file-write failure as well as the successful run. Small verified changes make faults easier to locate.

## Follow-up

Test discount boundaries, missing data, file failure and rollback behavior. Report which tests were actually run and which need the target system. Review OO-07, OO-13, TEST-01 and TEST-10.

# Exercise 11 Recover from an uncertain HTTP result

Your ABAP caller sends a create-order request with JSON. The service commits the order, but the connection times out before the response arrives. The retry job proposes sending the same request again with no correlation identifier.

**Your answer before reading the solution**

## Model answer

Do not assume the timeout means creation failed. The service may already have saved the order. Look it up using a stable business identifier, or use the API's supported idempotency key so repeating the same request does not create a second order.

Before retrying, establish the original outcome as far as the API allows. Check the response status and error body when a response does arrive. Send the required Content-Type and preserve the payload's meaning: a customer number such as 00123 may need to stay a string. An omitted field, JSON null and an empty string can request different behavior.

## Follow-up

What if the service returns 202? Explain how completion is observed. Explain why converting the JSON body to XML is a contract change, not just a header change. Review HTTP-01 through HTTP-12 and BTP-07.

# Exercise 12 Make a parallel background job restartable

A classic background job sends three independent work partitions through aRFC. Worker A commits, B fails and C is still running when the coordinator times out. An operator wants to rerun the whole job.

**Your answer before reading the solution**

## Model answer

Give each partition a stable ID and record what it actually completed. Worker A's committed changes remain even if the coordinator rolls back. Check B's failure and find out whether C finished after the timeout before rerunning either one.

Collect callbacks and RFC exceptions, and save checkpoints under a clear transaction design. Make unfinished work safe to repeat without duplicating completed business changes. Limit the number of workers to available resources and avoid sending overlapping locked records to different workers. Also check how the chosen WAIT and task-processing statements affect the database LUW. A finished job status alone does not prove every business item succeeded.

## Follow-up

Distinguish the job scheduler from parallel execution. Explain the roles of SM36/SM37 in a classic landscape and application-job templates in supported cloud scenarios. State why job status and business item success are separate evidence. Review RFC-05 through RFC-12.

# Exercise 13 Diagnose an IDoc or AIF error before resubmission

An inbound integration reports a mapping or application error. The business user sees the message in a configured AIF monitor. A similar earlier message already created a business document. An operator proposes repeated resubmission until the status turns green.

**Your answer before reading the solution**

## Model answer

Find the incoming message, its external business key and any document already created for that key. Check the IDoc control, data and status records, then the processing details shown by the configured AIF monitor. This tells you whether transport, mapping or business posting failed.

Fix the cause with the required permission, then use the mechanism that owns reprocessing. Before resubmitting, make sure the earlier message has not already produced the same business effect. Keep a record of the diagnosis and retry. AIF helps monitor and handle errors; it does not replace the underlying IDoc or OData processing rules.

## Follow-up

Explain how the diagnosis changes for transport hand-off versus application posting failure. Distinguish OData service semantics, IDoc asynchronous message processing and AIF monitoring/error handling. Review INTF-01 through INTF-12.

# A 60 minute mock interview

| Time | Interviewer prompt | What a strong response includes |
|---|---|---|
| 0 to 5 minutes | Introduce yourself and one real project | Your actual role, constraint and outcome |
| 5 to 12 minutes | Choose table types and explain a missing lookup | Key semantics, mutation and error handling |
| 12 to 20 minutes | Diagnose a slow report | Evidence, SQL volume, access pattern and correctness |
| 20 to 32 minutes | Design a request BO with draft and approval | Composition, transaction owner, messages and concurrency |
| 32 to 40 minutes | Explain Clean Core levels and a missing API | Target edition, classification, options and governance |
| 40 to 48 minutes | Discuss an OData or workflow failure | Protocol, retries, atomicity and security |
| 48 to 55 minutes | Explain testing and deployment | Real evidence, negative tests, dependencies and limits |
| 55 to 60 minutes | Ask questions about the role | Landscape, ownership, quality process and expectations |

# Score the interview

Score eight technical responses from 0 to 3. A 0 is wrong or missing; 1 is a definition with important gaps; 2 is correct with an example; 3 adds a relevant trade-off, failure mode and validation approach. Treat the score as a self-assessment rather than a hiring prediction. Choose the three weakest topics and repeat different questions from them tomorrow.

# Personal story worksheet

For each of five real examples, write: business problem; my responsibility; constraints; options considered; decision; testing evidence; result; what I learned. Use this for a delivered feature, a defect, a performance problem, an integration and a disagreement about design. Use only outcomes and measurements that actually occurred.

# Extension placement reference

## Choose an extension approach

Start with the business requirement, supported contracts and transaction boundary

| | | |
|---|---|---|
| **First** | **Can standard functionality or configuration meet the need?** | Use the standard capability and avoid unnecessary custom code. |
| **Key user** | **Does a delivered business context cover the adaptation?** | Use supported custom fields, logic or UI adaptation tools. |
| **On-stack** | **Does the logic need local transactional consistency?** | Use released local contracts and suitable ABAP Cloud extensibility. |
| **Side-by-side** | **Does the process need an independent or cross-system app?** | Use supported remote APIs and events; design retries and consistency. |

**Placement does not determine the Clean Core level. Check dependencies separately.**

# Study plan

The main route is ABAP Cloud, RAP and Clean Core: 154 questions, including 96 senior scenarios. Allocate about 70% of preparation time to this track and 30% to supporting ABAP and integration. Use the topic library to understand each answer with its pictures, then switch to the separate practice mode to recall it without reading ahead.

# Start with a senior diagnostic

Try CLOUD-01, CLOUD-04, SRAP-03, SRAP-08, SRAP-12, MODEL-01, MODEL-04, MODEL-14, OPS-05, OPS-11, GOV-02 and GOV-10. For each, explain the constraint, decision, rejected alternative and verification evidence. Score 0 for missing, 1 for partial, 2 for correct with an example, and 3 for a defensible trade-off with evidence. The two follow-up probes test whether your explanation survives a changed constraint.

A correct definition alone is not a senior answer. Mark a question ready only after explaining its failure boundary and a meaningful test. Review weak questions after one day, three days and one week. The browser records self-assessment; Anki supplies scheduled repetition if desired.

# Use the web studio

Open `START_HERE.html`. Learn starts with the complete topic catalog. Select a topic or Browse all questions. Topic icons, a breadcrumb and the sidebar show your current subject. Filter this view by text or ID narrows the current selection; topic/subtopic filters and the naturally sorted question index provide other ways to browse. All difficulty levels stay together without badges on question cards.

Show all answers and Questions only display a check and dark background on the selected option. Your display preference is remembered locally. Open or close individual answers as needed; a mixture of open and closed answers is labeled Custom view. Compact source links sit directly beneath each question. Diagrams have a captioned preview and numbered switcher; Enlarge diagram opens zoom, the original file and PNG download. Senior criteria and probes expand under Go deeper.

Use Search all topics and questions, or Ctrl+K / Cmd+K, to search all 329 questions, answers, examples and source titles. This search ignores scoped topic, difficulty and practice filters. A result opens the question with controls to return to the results or view it in its topic. Browser Back and Forward follow your navigation.

Practice uses one session builder and defaults to Interview practice across all questions and levels. Choose a subject, All questions / New questions / Retry mistakes, and a session of 5, 10, 20 or all available questions. More settings contains optional difficulty and search filters. New means no attempt in the selected format; retry uses the latest incorrect quiz attempt or spoken self-rating of 0 or 1.

Interview practice covers all 329 questions: explain the answer aloud, reveal the model answer, then give yourself a score of 0-3. Multiple-choice quiz covers the 103 questions with authored choices and explains all four options after you submit. For RAP, the coverage is 52 interview questions and 24 multiple-choice questions. The format cards make this distinction visible before you start. Quiz XP and spoken readiness are tracked separately; no microphone recording or automatic spoken grading is used.

During a session, global search and opening a question preserve the session in memory. Resume practice returns to the same fixed question list and current position. This works within the same page; reload does not preserve an unfinished session.

Skip for now leaves the question unattempted. Finish session shows results for answers submitted so far. Results separate mistakes or weak self-ratings from questions not attempted; quiz accuracy uses submitted answers only. Retry unsuccessful answers, practice the unattempted questions, or choose New session. Practice these questions transfers a library selection into Interview practice. Pictures and references appear with the relevant model answers and quiz feedback. Downloads & progress contains optional PDFs, Anki decks and progress import/export. Export progress before moving browsers. See the usability and content review for the revision and verification record.

# A daily hour

1. Recall weak questions for 10 minutes. 2. Study one Cloud segment and its visuals for 20 minutes. 3. Work through a senior scenario without the model answer for 15 minutes. 4. Take a 10-minute self-check, naming the evidence you would collect. 5. Record the weakest IDs and one unresolved release-specific assumption for 5 minutes.

# If the interview is in 48 hours

| Session | Work | Evidence of readiness |
|---|---|---|
| Day 1 morning | Cloud architecture 16 and Clean Core 08/20 | Defend extension placement, API contracts and a bounded exception |
| Day 1 afternoon | RAP 05/06/17 | Trace buffer, save, numbering, draft, concurrency and failure recovery |
| Day 1 evening | CDS/security 04/18 | Diagnose a DCL leak, unsupported filter and wrong count |
| Day 2 morning | Production 19 and weak foundations | Design meaningful tests and an uncertain-remote-outcome recovery |
| Day 2 afternoon | Senior mock: 75 minutes plus review | Score all six cases and repair the lowest dimensions |
| Final review | Quick revision and truthful project stories | Give a 90-second decision with an example and evidence |

# Seven day route

| Day | Main work | Practice |
|---|---|---|
| 1 | 16 Cloud architecture; 08 Clean Core baseline | Placement, runtime versus model, C0/C1/C2 versus A-D |
| 2 | 05 RAP foundations; first half of 17 | Transaction/save ownership, bulk EML and field presence |
| 3 | 06 advanced RAP; remaining 17 | Draft resume, entity/total ETags, cross-BO constraints |
| 4 | 04 CDS; 18 security and queries | Proposed-state checks, DCL entry point, count/paging/filter |
| 5 | 19 operations; 20 governance | Real BO tests, incidents, compatibility and exception retirement |
| 6 | Foundations 01/21, OO 02/22, patterns 23; integration by diagnostic need | HTTP/REST/JSON/XML, OData, RFC/BAPI/LUWs/jobs, IDoc/AIF |
| 7 | Six-case senior mock; 12 real project stories | Repair weak answers and explain two personal trade-offs |

# Fourteen day route

| Days | Focus | Output |
|---|---|---|
| 1-2 | Cloud architecture and Clean Core levels | Capability matrix and extension decision |
| 3-5 | RAP foundations and senior transactions | Save/concurrency drawings and three failure tests |
| 6-7 | CDS, authorization and custom queries | Security matrix and query-contract test cases |
| 8-9 | Production, testing and performance | Evidence plan and incident recovery walk-through |
| 10 | Clean Core governance | Bounded exception and upgrade/retirement plan |
| 11-12 | Foundations 21, OO 22, patterns 23 and delivery/HANA 24 | Trace value/reference behavior, defend a pattern and verify integration recovery |
| 13 | Senior mock without notes | Six scored cases and weakest three competencies |
| 14 | Weak cards, quick revision and project stories | Two-minute truthful examples with measured outcomes |

Revisit earlier mistakes on days 4, 8 and 12. Read baseline modules when the senior scenario exposes a gap; do not replace missing foundations with memorized architecture phrases.

# Rehearse truthful project stories

Prepare a two-minute story for a delivered feature, a production defect, a performance improvement, a difficult integration and a design trade-off. For each, record the business context, your role, constraint, decision, verification and outcome. Use your actual projects and measurements. Clearly distinguish a learning exercise from production delivery.

# Import the flashcards

Import `flashcards/ABAP_Interview_Anki.tsv` into Anki as UTF-8, tab-separated text. The file has no header row. Map column 1 to Front, column 2 to Back and column 3 to Tags; enable HTML in fields. Use a Basic note type. Question IDs are included in the front so updates can be matched. The HTML Study Desk also works without Anki: download the repository and open `START_HERE.html` locally.

# Release context to clarify

Before discussing a concrete implementation, establish SAP edition, release and support package, ABAP language version, API release status, consumer and SAPUI5 version. Classic transactions in the guide are examples for landscapes where those tools are available. The date of this study pack is not a claim that every feature exists on every target system.

# Learn with visual explanations

In START_HERE.html, open a question's answer to see its linked visual previews. Select a numbered diagram and use Enlarge diagram to inspect it without leaving the question. Interview practice reveals diagrams with the model answer; Multiple-choice quiz makes them available under Full explanation & diagrams after checking. The GitHub gallery and printable atlas remain available for browsing the full collection; the app's Downloads & progress menu links to both.

For each picture, first describe what is happening without reading the caption. Answer its recall prompt, then state where the analogy or model stops being accurate. Redraw one example from memory: a join result, a stale ETag update, the RAP artifact stack or the Clean Core levels. Reopen the related questions to check the explanation.

# Illustrated Anki deck

Use `flashcards/ABAP_Interview_Illustrated_Anki.tsv` for the same complete question set with images. Copy the PNG files from `flashcards/media/` directly into your Anki profile's `collection.media` directory, preserving filenames. Import with Front / Back / Tags mapping and HTML enabled. Do not add a media subdirectory.

Choose one deck for the first import. Both use identical front fields, so use Anki's matching/update options when replacing an existing deck. Check the import preview and verify that `RAP-12` displays its images.

Reference: Anki text and media import manual.

# Focused senior Anki deck

Use `flashcards/ABAP_Cloud_Senior_Anki.tsv` for only the 96 senior scenarios. It includes follow-up probes, strong-answer criteria and verification evidence. It is a subset of both full decks and uses the same front fields; choose one initial import or use Anki matching/update options.

# Focused foundations and pattern route

Start with BASE-01/02/04/06/12/13/16, OODEEP-01/03/06/09/12 and PAT-01/03/04/06/08/09/12. Predict the worked example before reading it. For each pattern, name the variation or failure boundary it solves and the simpler alternative you rejected. Complete a challenge, explain all three distractors, then repeat the same decision aloud without options. Read the quiz guide and current market coverage to choose remaining gaps.